Part 1:

1-1.)

```
.global _start
_start:
      MOV R0 , #25 @ Load 25 into R0
      MOV R1 , #17 @ Load 17 into R1
      @ Your code here
      ADDS R2, R0, R1
      SUBS R3, R0, R1
      @ End program ( infinite loop )
done :
      B done
```

1-2.)

r0	00000019	
r1	00000011	
r2	0000002a	
r3	00000008	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000010	
cpsr	200001d3	NZCVI SVC
spsr	00000000	NZCVI ?

1-3.)

The program starts by moving the values 25 and 17 into r0 and r1. The program then adds those values together and stores the result in r2 and subtracts the values and stores the result in r3.

Part 2:

2-1.)

```
.global _start
_start:
            @setting values
             ldr r0, =#0xABCD1234
             mov r1, #0
             mov r2, #0
             mov r3, #0
loop:
            AND r1, r0, #1 @r1 = r0 & 1
            ADD r2, r2, r1 @r2 += r1
            LSR r0, #1 @r0 >> 1
            CMP r0, #0 @ while(r0 != 0)
            BNE loop
            mov r0, r4 @resetting input
            mov r1, r2 @setting output
done :
            B done
```

2-2.)

r0	abcd1234
r1	0000000f
r2	0000000f
r3	00000000
r4	abcd1234
r5	00000000
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000000
r11	00000000
r12	00000000
sp	00000000
lr	00000000
pc	00000030
cpsr	600001d3
spsr	00000000

2-3.)

The loop terminates by comparing the input to 0 each time it has been left shifted. Once it equals 0, meaning it has been left shifted all the way to the right and no more bits are left to check, the loop terminates.

Part 3:

3-1.)

```
.data
array : .word 15 , -3 , 42 , 8 , -17 , 23 , 4 , -9 , 31 , 0
length : .word 10

.text
.global _start
_start :
            LDR R0 , =array @ R0 = address of array
            LDR R1 , =length
            LDR R1 , [R1] @ R1 = length of array
            mov r2, #0 @index
            mov r3, #0 @array values
            mov r4, #0 @maximum
            mov r5, #0 @minimum
            mov r10, #4 @setting the index to compare and move the correct
amount
            mul r1, r1, r10
loop:
            ldr r3, [r0, r2] @r3 = array[r0 + r2]
            cmp r3, r4 @if (r3 > r4)
            BGT maximum
            cmp r3, r5 @if (r3 < r5)
            BLT minimum
comp:
            add r2, r2, #4 @r2 += 4
            cmp r2, r1 @while (r2 != r1)
            bne loop
done:
            b done
minimum:
            mov r5, r3 @r5 = r3
            b comp
maximum:
            mov r4, r3 @r4 = r3
            b comp
```

3-2.)

00000060	0000000f	fffffffd	0000002a	00000008
00000070	ffffffef	00000017	00000004	fffffff7
00000080	0000001f	00000000	0000000a	00000000

3-3.)

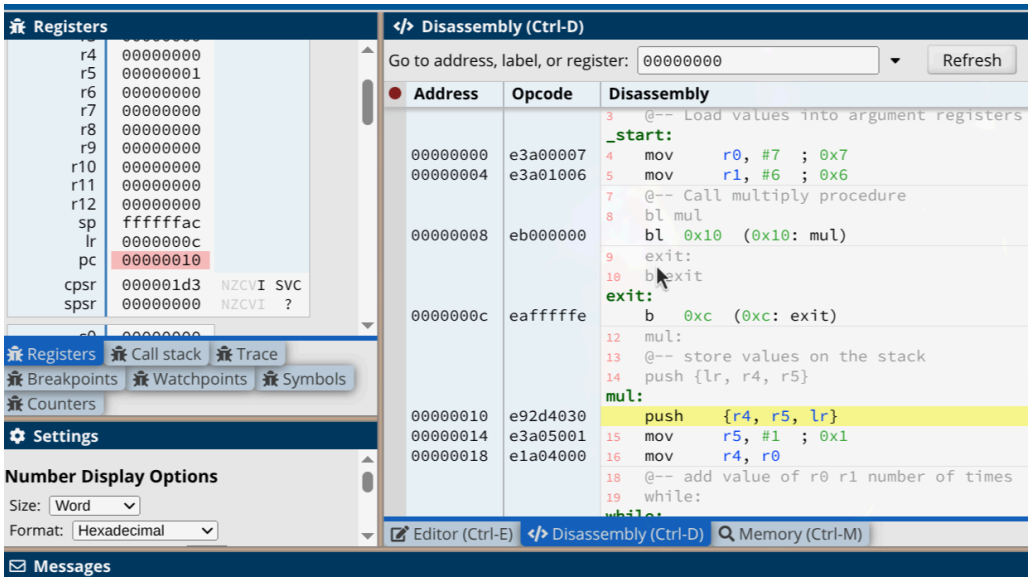
r0	00000060
r1	00000028
r2	00000028
r3	00000000
r4	0000002a
r5	ffffffef
r6	00000000
r7	00000000
r8	00000000
r9	00000000
r10	00000004
r11	00000000
r12	00000000
sp	00000000
lr	00000018
pc	00000044
cpsr	600001d3
spsr	800001d3

Part 4:

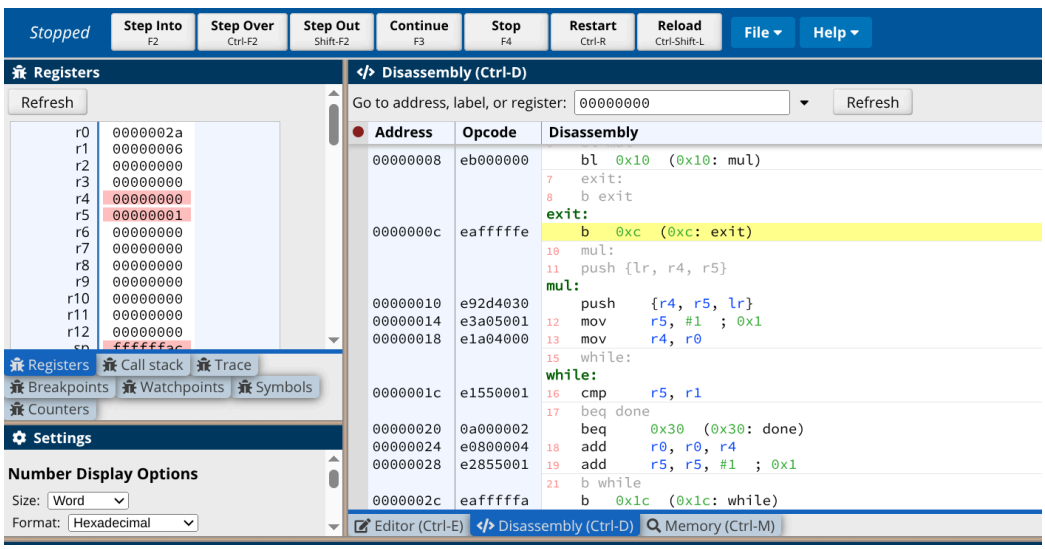
4-1.)

```
.global _start
_start:
      @-- Load values into argument registers
      mov r0, #7
      mov r1, #6

      @-- Call multiply procedure
      bl mul
exit:
      b exit

mul:
      @-- store values on the stack
      push {lr, r4, r5}
      mov r5, #1
      mov r4, r0

      @-- add value of r0 r1 number of times
while:
      cmp r5, r1
      beq done
      add r0, r0, r4
      add r5, r5, #1

      b while
done:
      @--restore values saved on the stack
      pop {pc, r4, r5}
  bx lr
```

4-2.)



4-3.)



R0 has the value 0000_002a which is the value 42.

4-4.)

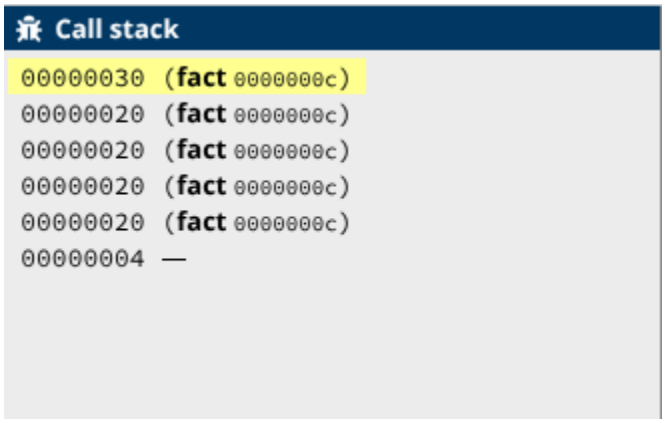
If another procedure was called from within multiply without saving lr on the stack you would not be able to get back to multiply when the new procedure returns.

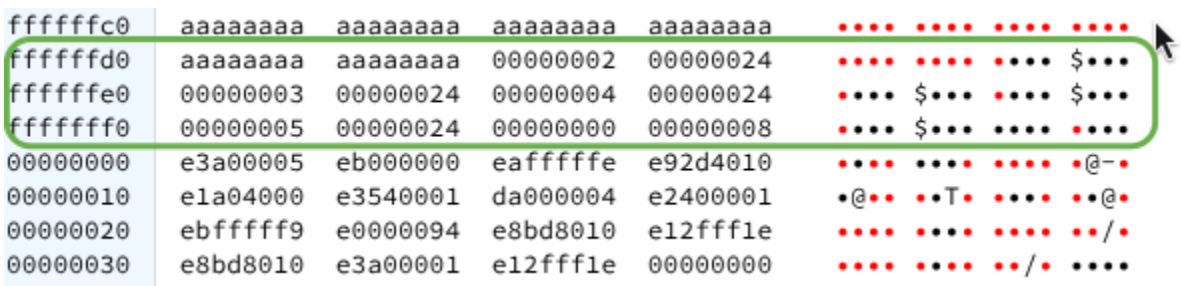
Part 5:

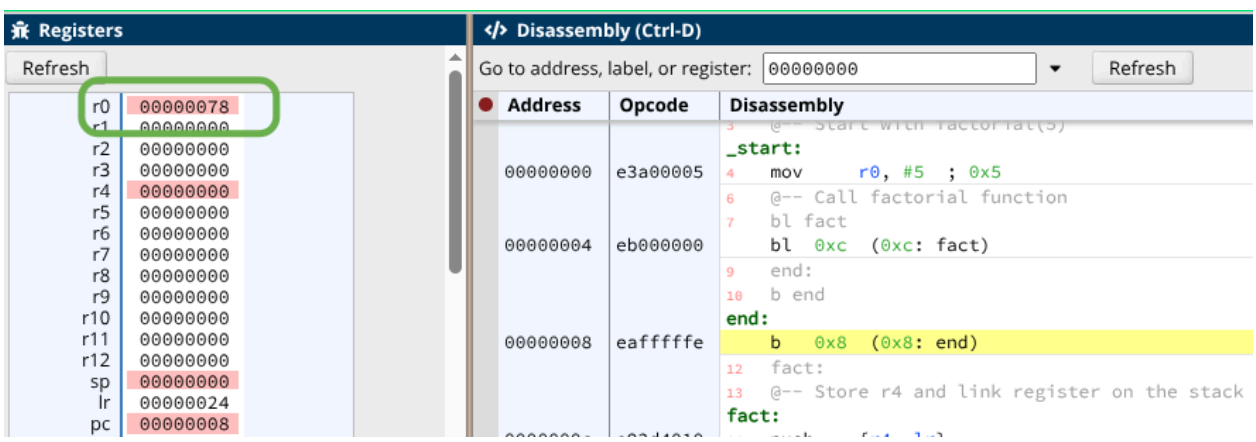
5-1.)

```
.global _start
_start:
      @-- Start with factorial(5)
      mov r0, #5

      @-- Call factorial function
      bl fact

end:
      b end

fact:
      @-- Store r4 and link register on the stack
      push {r4, lr}
      mov r4, r0
      cmp r4, #1
      ble base

      @-- n - 1
      sub r0, r0, #1

      @-- recursive call to factorial function
      bl fact

      @-- multiply n * result of factorial(n - 1)
      mul r0, r4, r0

      @-- restore values stored on the stack
      pop {r4, pc}
      bx lr


base:
      @-- base case of the recursive function return 1
      pop {r4, pc}
      mov r0, #1
      bx lr
```

5-2.)







Note: r0 is 0000_0078 which is 120 in decimal. PC is at 0000_0008 which is the address off the end: label in the disassembly on the right

5-3.)

```
                HIGH MEMORY
-------------------------------------------------
| fact(5) frame                                  |
| saved r4 = 5                                   |
| saved lr → return to _start                    |
-------------------------------------------------
| fact(4) frame                                  |
| saved r4 = 4                                   |
| saved lr → return to fact(5)                   |
-------------------------------------------------
| fact(3) frame                                  |
| saved r4 = 3                                   |
| saved lr → return to fact(4)                   |
-------------------------------------------------
| fact(2) frame                                  |
| saved r4 = 2                                   |
| saved lr → return to fact(3)                   |
-------------------------------------------------
| fact(1) frame  ← deepest point                 |
| saved r4 = 1                                   |
| saved lr → return to fact(2)                   |
-------------------------------------------------
                LOW MEMORY  (sp points here)
```

5-4.)

Call	N (R0)	LR before PUSH	SP after PUSH	Stack Contents
Main -> fact(4)	4	0000_0008	ffff_ffd0	0000_0008 lr to main 0000_0000 (r4)
fact(4) -> fact(3)	3	0000_0024	ffff_fff0	0000_0005 (r4) 0000_0024 lr to fact(4)
fact(3) -> fact(2)	2	0000_0024	ffff_ffe8	0000_0004 (r4) 0000_0024 lr to fact(3)
fact(2) -> fact(1)	1	0000_0024	ffff_ffe0	0000_0003 (r4) 0000_0024 lr to fact(2)

Note: mul is at 0000_0024 that is why lr is the same of each recursive call to fact.This is what causes each recursive call to pick back up at mul when pop pc occurs.

Returning From	Return value (R0)	SP after POP
fact(1)	0000_0001	ffff_ffe0
fact(2)	0000_0002	ffff_ffe8
fact(3)	0000_0006	ffff_fff0
fact(4)	0000_0018	ffff_fff8

5-5.)
The link register (LR) must be saved before making a recursive call because bl stores the return address in LR. In recursion, each call to the function overwrites LR with the return address for the current call. If LR is not saved on the stack, returning from the deepest call would restore the wrong address and all previous return addresses would be lost. Then the program would jump to an incorrect instruction, likely causing a crash or undefined behavior. By using push {LR}, each recursive call preserves its own return address on the stack, ensuring that when the function returns, execution resumes correctly in the caller.

Part 6:

6-1.)

```
.data
message:    .asciz "Hello World"
shift:            .word 3

.text
.global _start
_start:
      LDR   R2, =message            @ Dereference data
      LDR   R3, =shift
      LDR   R6, [R3]
decrease:
      CMP         R6, #26     @ Get the shift val down to less than 27
      BLT         continue
      SUB         R6, R6, #26
      B           decrease
continue:
      MOV   R4, #0;                 @ Use R4 to hold string offset
for:
      LDRB  R0, [R2, R4]            @ Load char

      CMP         R0, #0            @ Jump to end when no more chars
      BEQ         _stop

CMP     R0, #'A'        @ Verify the char is A-Z or a-z
BLT     end_of_for
CMP     R0, #'Z'
BLE     run

CMP     R0, #'a'
BLT     end_of_for
CMP     R0, #'z'
BLE     run

B       end_of_for

run:
      BL          encrypt     @ Perform encoding or decoding on char
@ To switch from encryption to decryption, you simply have to switch
"encrypt"
@ for "decrypt". It was unclear exactly how you wanted me to show both
```

```
@ procedures in the same code but this is what I guessed you wanted.

STRB  R0, [R2, R4]            @ Save the char in its location

end_of_for:                   @ Continue the loop
      ADD         R4, R4, #1
      B           for

_stop:
    B _stop


encrypt:
    CMP     R0, #'a'          @ Check if the letter is upper or lower case
    BGE     lower_case

    ADD     R0, R0, R6        @ Handle the uppercase case
    CMP     R0, #'Z'
    BLE     done
    SUB     R0, R0, #26
    B       done

lower_case:             @ Handle the lowercase case
    ADD     R0, R0, R6
    CMP     R0, #'z'
    BLE     done
    SUB     R0, R0, #26

done:
    BX      LR

decrypt:
    CMP     R0, #'a'          @ Check if letter is upper or lower
    BGE     de_lower_case

    SUB     R0, R0, R6        @ Handle uppercase case
    CMP     R0, #'A'
    BGT     de_done
    ADD     R0, R0, #26
    B       de_done

de_lower_case:          @ Handle lowercase case
    SUB     R0, R0, R6
```

```
    CMP     R0, #'a'
    BGT     de_done
    ADD     R0, R0, #26

de_done:
    BX      LR
```

6-2.)

```
000000c0   e12fff1e   000000d0   000000dc   00000000    ••/• •••• •••• ••••
000000d0   6c6c6548   6f57206f   00646c72   00000003    Hell o Wo rld• ••••
000000e0   aaaaaaaa   aaaaaaaa   aaaaaaaa   aaaaaaaa    •••• •••• •••• ••••
```

6-3.)

```
000000c0   e12fff1e   000000d0   000000dc   00000000    ••/• •••• •••• ••••
000000d0   6f6f684b   725a2072   00676f75   00000003    Khoo r Zr uog• ••••
000000e0   aaaaaaaa   aaaaaaaa   aaaaaaaa   aaaaaaaa    •••• •••• •••• ••••
```

6-4.)

```
000000c0   e12fff1e   000000d0   000000dc   00000000    ••/• •••• •••• ••••
000000d0   6c6c6548   6f57206f   00646c72   00000003    Hell o Wo rld• ••••
000000e0   aaaaaaaa   aaaaaaaa   aaaaaaaa   aaaaaaaa    •••• •••• •••• ••••
```

6-5.)

To handle the wrap around case, I made sure that the shift value was less than 27 so that if I did need to wrap around, I would only have to do so once. Next, I determined if the value was in the range of uppercase letters or lowercase letters. Depending on if the char was in the upper case or lower case range, I next added the shift value to whatever the char was. If the value that I got was either greater than ‘Z’ or ‘z’ depending on if it was upper or lower case, I then subtracted 26 from the value to wrap around back to the front of the alphabet.